

CMPT 354
Database Systems I

Relational Algebra

Martin Ester
Simon Fraser University
Fall 2023

Relational Query Languages



- *Query languages*: Allow manipulation and **retrieval of data** from a database.
- Relational model supports simple, powerful query languages:
 - Strong formal foundation based on logic.
 - High level, abstract formulation of queries.
 - Easy to program.
 - Allows the DBS to do much optimization.
- DBS can choose, e.g., most efficient sorting algorithm or the order of basic operations.

Relational Query Languages



- Query Languages **!=** programming languages!
 - QLs not expected to be “Turing complete”.
 - QLs not intended to be used for complex calculations.
 - QLs support easy, efficient access to large data sets.
- E.g., in a QL cannot
 - determine whether the number of tuples of a table is even or odd,
 - create a visualization of the results of a query,
 - ask the user for additional input.

Formal Query Languages



- Two mathematical query languages form the basis for “real” languages (e.g. SQL), and for implementation:
 - *Relational Algebra (RA)*: More *procedural*, very useful for representing execution plans, relatively close to SQL.
 - *Relational Calculus (RC)*: Lets users describe what they want, rather than how to compute it. (*Non-procedural, declarative.*)
- Understanding these formal query languages is important for understanding SQL and query processing.

Algebra



- An algebra consists of *operators* and *operands*. Operands can be either variables or constants.
- In the algebra of arithmetic, atomic operands are variables such as x or y and constants such as 15. Operators are the usual arithmetic operators such as $+$, $-$, $*$.
- *Expressions* are formed by applying operators to atomic operands or other expressions.
- For example,
 - 15
 - $x + 15$
 - $(x + 15) * y$

Algebra



- Algebraic expressions can be re-ordered according to commutativity or associativity laws without changing their resulting value.

- E.g.,

$$15 + 20 = 20 + 15$$

$$(x * y) * z = x * (y * z)$$

- Parentheses group operators and define precedence of operators, e.g.

$$(x + 15) * y$$

$$x + (15 * y)$$

Relational Algebra



- In relational algebra, *operands* are relations / tables, and an expression evaluates to a relation / set of tuples.
- The relational algebra *operators* are
 - set operations,
 - operations removing rows (selection) or columns (projection) from a relation,
 - operations combining two relations into a new one (Cartesian product, join),
 - a renaming operation, which changes the name of the relation or of its attributes.

Relational Algebra



- A query is applied to *relation instances*, and the result of a query is also a relation instance.
 - *Schemas of input* relations for a query are *fixed* (but query will run regardless of instance!)
 - The *schema for the result* of a given query is also *fixed*! Determined by definition of input relations and query language constructs.
- Positional vs. named-attribute notation:
 - Positional notation easier for formal definitions.
 - Named-attribute notation more readable.

Example Instances



- “Sailors” and “Reserves” relations for our examples.
- We’ll use positional or named attribute notation, assume that names of attributes in query results are ‘inherited’ from names of attributes in query input relations.

R1

<u>sid</u>	<u>bid</u>	<u>day</u>
22	101	10/10/96
58	103	11/12/96

S1

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

Relational Algebra Operations



■ Basic operations

- *Selection* (σ)
Selects a subset of *rows* from relation.
- *Projection* (π)
Deletes unwanted *columns* from relation.
- *Cartesian product* ($\times$)
Combine *two relations*.
- *Set-difference* ($-$)
Tuples in relation 1, but *not in* relation 2.
- *Union* ($\cup$)
Tuples in relation 1 *or* in relation 2.

Relational Algebra Operations



- Renaming of relations / attributes.
- Additional operations:
 - Intersection, join, division.
 - Not essential, can be implemented using the five basic operations.
 - But (very!) useful.
- Since each operation returns a relation, *operations can be composed*, i.e. output of one operation can be input of the next operation.
- Algebra is closed!

Renaming



- Renames relations / attributes, without changing the relation instance.

- $\rho^{S(A_1, A_2, \dots, A_n)}(R)$

relation R is renamed to S,
attributes are renamed $A_1, \dots, A_n$

- Rename only some attributes

$$\rho^{S(1 \rightarrow A_1, \dots, k \rightarrow A_k)}(R)$$

using the positional notation to reference attributes

- No renaming of attributes

$$\rho^S(R)$$

Projection



- One input relation.
- Deletes attributes that are not in *projection list*.
- *Schema* of result contains exactly the attributes in the projection list, with the same names that they had in the (only) input relation.
- Projection operation has to eliminate *duplicates*, since relations are sets.
- Duplicate elimination is expensive.
- Therefore, commercial DBMS typically don't do duplicate elimination unless the user explicitly asks for it.

Projection



S2

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sname	rating
yuppy	9
lubber	8
guppy	5
rusty	10

$\pi_{sname, rating}(S2)$

age
35.0
55.5

$\pi_{age}(S2)$

Selection



- One input relation.
- Selects all tuples that satisfy *selection condition*.
- No duplicates in result!
(Why?)
- *Schema* of result identical to schema of (only) input relation.
- Selection conditions:
 - *simple conditions* comparing attribute values (variables) and / or constants or
 - *complex conditions* that combine simple conditions using logical connectives AND and OR.

Selection



S2

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

$\sigma_{rating > 8}(S2)$

sid	sname	rating	age
28	yuppy	9	35.0
58	rusty	10	35.0

$\pi_{sname, rating}(\sigma_{rating > 8}(S2))$

sname	rating
yuppy	9
rusty	10

Union, Intersection, Set-Difference



- All of these set operations take two input relations, which must be *union-compatible*:
 - Same sets of attributes.
 - Corresponding attributes have same type.
- What is the *schema* of result?

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0
44	guppy	5	35.0
28	yuppy	9	35.0

$S1 \cup S2$

sid	sname	rating	age
22	dustin	7	45.0

$S1 - S2$

sid	sname	rating	age
31	lubber	8	55.5
58	rusty	10	35.0

$S1 \cap S2$

Cartesian Product



- Also referred to as *cross-product* or *product*.
- Two input relations.
- Each tuple of the one relation is paired with each tuple of the other relation.
- *Result schema* has one attribute per attribute of both input relations, with attribute names 'inherited' if possible.
- In the result, there may be two attributes with the same name, e.g. both S1 and R1 have an attribute called *sid*.
- Then, apply the *renaming operation*, e.g.

Cartesian Product



R1

S1

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

<u>sid</u>	<u>bid</u>	<u>day</u>
22	101	10/10/96
58	103	11/12/96

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

S1 × *R1*

Join



- Similar to Cartesian product with same result schema.
- Each tuple of the one relation is paired with each tuple of the other relation if the two tuples satisfy the *join condition*.

■ *Theta-Join*: $R \bowtie_c S = \sigma_c (R \times S)$

Example: $S1 \bowtie_{S1.sid < R1.sid} R1$

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	58	103	11/12/96

Join



- *Equi-Join*: A special case of Theta-join where the condition c contains only *equalities*.

Example: $S1 \bowtie_{sid} R1$

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96

- *Result schema* similar to Cartesian product, but only one copy of attributes for which equality is specified.
- *Natural Join*: Equi-join on *all* common attributes. Example: $S1 \bowtie R1$

Division



- Not supported as a primitive operation, but useful for expressing queries like:
*Find sailors who have reserved **all** boats.*
- Let A have 2 attributes, x and y ; B have only attribute y :
 - $A/B = \{x \mid \forall y \in B: \exists \langle x, y \rangle \in A\}$
 - i.e., A/B contains all x tuples (sailors) such that for **every** y tuple (boat) in B , there is an xy tuple (reservation) in A .
- In general, x and y can be any lists of attributes; y is the list of attributes in B , and $x \cup y$ is the list of attributes of A .

Division



sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

A

pno
p2

B1

sno
s1
s2
s3
s4

A/B1

pno
p2
p4

B2

sno
s1
s4

A/B2

pno
p1
p2
p4

B3

sno
s1

A/B3

Division



- Division is not an essential operation; can be implemented using the five basic operations.
 - Also true of joins, but joins are so common that systems implement joins specially.
- *Idea*: For A/B , compute all x values in A that are not 'disqualified' by some y value in B .
 - x value in A is *disqualified* if by attaching y value from B , we obtain an xy tuple that is not in A .

Disqualified x values: $\pi_x((\pi_x(A) \times B) - A)$

A/B : $\pi_x(A) -$ all disqualified x values

Formulating RA Queries



- To formulate an RA query, ask the following questions.
- Which relations do we need?
 - joins
- Which tuples do we need?
 - selections
- Which attributes do we need?
 - projections

Formulating RA Queries



- The renaming operation can be used to decompose a complex query into multiple steps, which may make it easier to understand.
- Example
names of sailors who have reserved a red boat

$$\begin{aligned} &\rho_{BidsRedboats}(\pi_{bid}(\sigma_{color=red}Boats)) \\ &\rho_{SidsReservedRedboats}(\pi_{sid}(Reserves \bowtie \\ &BidsRedboats)) \\ &\pi_{sname}(Sailors \bowtie SidsReservedRedboats) \end{aligned}$$

Example Queries



- Use additional table Boats:

Boats(bid: string, *bname*: string, *color*: string)

primary key underlined

Example Queries



- Find names of sailors who've reserved boat #103.
- Solution 1:
- Solution 2:
- Solution 3:

Example Queries



- Find names of sailors who've reserved a red boat.
- Information about boat color only available in Boats; so need an extra join.
- A more efficient solution:

Example Queries



- Find names of sailors who've reserved a red *or* a green boat.
- Can identify all red or green boats, then find sailors who've reserved one of these boats:
- Using UNION instead of OR:

Example Queries



- Find sailors who've reserved a red *and* a green boat.
- Previous approaches won't work!
- Why?

Example Queries



- Must identify sailors who've reserved red boats, sailors who've reserved green boats, then find the intersection:

Example Queries



- Find the names of sailors who've reserved *all* boats.
- Uses division; schemas of the input relations must be carefully chosen:

Example Queries



- To find sailors who've reserved all boats with bname 'Interlake' :

Example Queries



- Find the names of sailors who've reserved a (at least one) boat:

Example Queries



- Find the names of sailors who've reserved no boat:

Query Optimization



- A user of a commercial DBMS formulates SQL queries.
- The query optimizer translates this query into an *equivalent* RA query, i.e. an RA query with the same result.
- In order to optimize the efficiency of query processing, the query optimizer can re-order the individual operations within the RA query.
- Re-ordering has to preserve the query semantics and is based on RA equivalences.

Query Optimization



- Why can re-ordering improve the efficiency?
- Different orders can imply different sizes of the intermediate results.
- The smaller the intermediate results, the more efficient.

- Example:

$((\sigma_{color='red'} Boats) \bowtie Reserves) \bowtie Sailors$

much (!) more efficient than

$\sigma_{color='red'}((Reserves \bowtie Sailors) \bowtie Boats)$

Relational Algebra Equivalences



- The most important RA equivalences are commutative and associative laws.
- A *commutative law* about some operation states that the order of (two) arguments does not matter.
- An *associative law* about some (binary) operation states that (more than two) arguments can be grouped either from the left or from the right.
- If an operation is both commutative and associative, then any number of arguments can be (re-)ordered in an arbitrary manner.

Relational Algebra Equivalences



- The following (binary) RA operations are commutative and associative: $\times$ $\bowtie$ $\cup$ $\cap$
- For example, we have:

$$(R \bowtie S) \equiv (S \bowtie R) \quad (\text{Commutative})$$

$$R \bowtie (S \bowtie T) \equiv (R \bowtie S) \bowtie T \quad (\text{Associative})$$

- Proof method: show that each tuple produced by the expression on the left is also produced by the expression on the right and vice versa.

Relational Algebra Equivalences



- Selections are crucial from the point of view of query optimization, because they typically reduce the size of intermediate results by a significant factor.
- Laws for selections only:

$$\sigma_{c1 \text{ AND } \dots \text{ AND } cn}(R) \equiv \sigma_{c1}(\dots \sigma_{cn}(R)) \quad (\textit{Splitting})$$

$$\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R)) \quad (\textit{Commutative})$$

Relational Algebra Equivalences



- Laws for the combination of selections and other operations:

if R has all attributes mentioned in c

$$\sigma_c(R \times S) \equiv \sigma_c(R) \times S$$

if S has all attributes mentioned in c

$$\sigma_c(R \times S) \equiv R \times \sigma_c(S)$$

- The above laws can be applied to “push selections down” as much as possible in an expression, i.e. performing selections as early as possible.

Relational Algebra Equivalences



- A projection commutes with a selection that only uses attributes retained by the projection.
- Selection between attributes of the two arguments of a Cartesian product converts Cartesian product to a join.
- Similarly, if a projection follows a join $R \bowtie S$, we can 'push' it by retaining only attributes of R (and S) that are needed for the join or are kept by the projection.

Summary



- The relational model has formal query languages that are easy to use and allow efficient optimization by the DBS.
- Relational algebra (RA) is more procedural; used as internal representation for SQL query evaluation plans.
- Five basic RA operations: selection, projection, Cartesian product, union, set-difference.
- Additional operations as shorthand for important cases: intersection, join, division.
- These operations can be implemented using the basic operations.

Summary



- Several ways of expressing a given query; a query optimizer chooses the most efficient version.
- Query optimization exploits RA equivalencies to re-order the operations within an RA expression.
- Optimization criterion is to minimize the size of intermediate relations.